FIAP - Faculdade de Informática e Administração Paulista
Pós-graduação em Inteligência Artificial para Desenvolvedores

Gustavo Feikes Marino de Siqueira

OTIMIZAÇÃO GENÉTICA E EXPLICAÇÕES EM LINGUAGEM NATURAL PARA APOIO AO DIAGNÓSTICO DE CÂNCER DE MAMA

Relatório técnico do Tech Challenge da Fase 2

Relatório técnico apresentado como requisito de avaliação do Tech Challenge da Fase 2 do curso de Pós-graduação em Inteligência Artificial para Desenvolvedores da FIAP.

São Paulo
2026

RESUMO

Este relatório documenta a Fase 2 do sistema de apoio ao diagnóstico de câncer de mama construído na Fase 1 sobre a base Breast Cancer Wisconsin (Diagnostic). A Fase 2 acrescenta três camadas: um algoritmo genético que otimiza conjuntamente o subconjunto de medidas, os hiperparâmetros e o limiar de decisão; uma camada de modelo de linguagem que transforma cada previsão em explicação para o médico, com verificação automática da qualidade; e o sistema servido como API monitorada, com painel, vitrine interativa e publicação gratuita em nuvem. A função fitness é o custo clínico assimétrico, em que o falso negativo pesa 10 vezes o falso positivo, medido por validação cruzada dentro do conjunto de treino. Foram 45 buscas, em 4 configurações do algoritmo, 3 famílias de modelo e 3 sementes, mais braços de ablação que isolam de onde vem o ganho. O achado central é que a redução de falsos negativos vem de mover o ponto de operação, pelo limiar ou pelo peso das classes, e não da seleção de medidas: no conjunto de teste, os braços que só ajustam o limiar ou só os hiperparâmetros chegam a 1 falso negativo, contra 3 da referência da Fase 1. O modelo servido, escolhido por uma regra registrada antes de olhar o teste, usa 5 das 30 medidas e empata com a referência em falsos negativos. Os textos são gerados pelo Claude Haiku 5.5, em lote: 116 de 116 passaram nas quatro verificações automáticas, a um custo total de US$ 0,046. Conclui-se que o algoritmo genético é a ferramenta certa para o espaço combinatório que a Fase 1 deixou aberto, e que o resultado honesto é mais modesto que o esperado: menos medidas sem perder segurança, mas não mais segurança por escolher medidas.

Palavras-chave: algoritmos genéticos; modelos de linguagem; apoio ao diagnóstico; custo clínico; explicabilidade.

SUMÁRIO

RESUMO 2

1 INTRODUÇÃO E OBJETIVOS 5

2 ARQUITETURA DA SOLUÇÃO 6

3 IMPLEMENTAÇÃO DO ALGORITMO GENÉTICO 7

3.1 Codificação do indivíduo 7

3.2 Função fitness 7

3.3 Operadores 8
3.4 Experimentos 8
3.5 Braços de ablação 9
3.6 Execução 9
4 RESULTADOS DA OTIMIZAÇÃO 10
4.1 Convergência 10
4.2 De onde vem o ganho 11
4.3 Verificação de estabilidade 11
4.4 Seleção de medidas e convergência com a Fase 1 12
4.5 Fronteira entre custo e número de medidas 13
4.6 Sensibilidade à política de custo 14
5 COMPARAÇÃO ENTRE MODELOS ORIGINAIS E OTIMIZADOS 15
5.1 Regra de escolha registrada antes do teste 15
5.2 Resultados no conjunto de teste 15
5.3 Leitura honesta do resultado 16
6 INTEGRAÇÃO COM MODELO DE LINGUAGEM 16
6.1 Abordagem 16
6.2 Técnicas de prompt engineering 17
6.3 Avaliação automática da qualidade 17
6.4 Comparação entre versões de prompt 18
6.5 Exemplo e avaliação humana 19
7 SERVIÇO, MONITORAMENTO E ESCALABILIDADE 20
7.1 API 20
7.2 Logging e monitoramento 21
7.3 Escalabilidade 22
7.4 Vitrine e painel 23
8 IMPLANTAÇÃO EM NUVEM 23

9 DESAFIOS ENFRENTADOS E SOLUÇÕES 24
10 REPRODUTIBILIDADE E TESTES 24
11 CONSIDERAÇÕES FINAIS 25
REFERÊNCIAS 25

1 INTRODUÇÃO E OBJETIVOS

A Fase 1 entregou um classificador de tumores de mama com regressão logística, recall de 0,929 na classe maligna e 3 falsos negativos nos 114 casos de teste, explicado por coeficientes, importância por permutação e SHAP. O enunciado da Fase 2 oferece dois projetos; este trabalho segue o Projeto 1, otimização de modelos de diagnóstico, porque dá continuidade direta ao Módulo 1 e reaproveita toda a infraestrutura já construída.

O enunciado pede três entregas técnicas: otimizar por algoritmo genético os modelos da Fase 1 e compará-los com os originais; servir o sistema com monitoramento, logging e preparo para escalabilidade; e integrar um modelo de linguagem que explique os diagnósticos, transforme números em orientações acionáveis e prepare a base para os dados textuais do Módulo 3.

Existe uma armadilha nesse enunciado, e o desenho do trabalho nasce de reconhecê-la. A busca em grade da Fase 1 já tinha saturado o espaço de hiperparâmetros: a regressão logística ficou no valor padrão `C = 1,0`, e o F1 de validação não se moveu. Um algoritmo genético aplicado ao mesmo espaço produziria empate técnico, e a entrega teria de escolher entre inflar um ganho na terceira casa decimal ou admitir que não fez nada.

Busca em grade e algoritmo genético não competem no mesmo terreno. A grade serve a poucos parâmetros contínuos; o algoritmo genético é insubstituível no espaço combinatório. A Fase 1 deixou dois desses espaços abertos, ambos registrados no próprio relatório:

- **seleção de medidas:** são 30 medidas, mais de um bilhão de subconjuntos, com 21 pares de correlação acima de 0,9 que a Fase 1 decidiu não remover;
- **limiar de decisão:** a Fase 1 escolheu o limiar 0,30 olhando o conjunto de teste, classificou isso como fragilidade metodológica e listou a calibração por validação cruzada no treino como próximo passo.

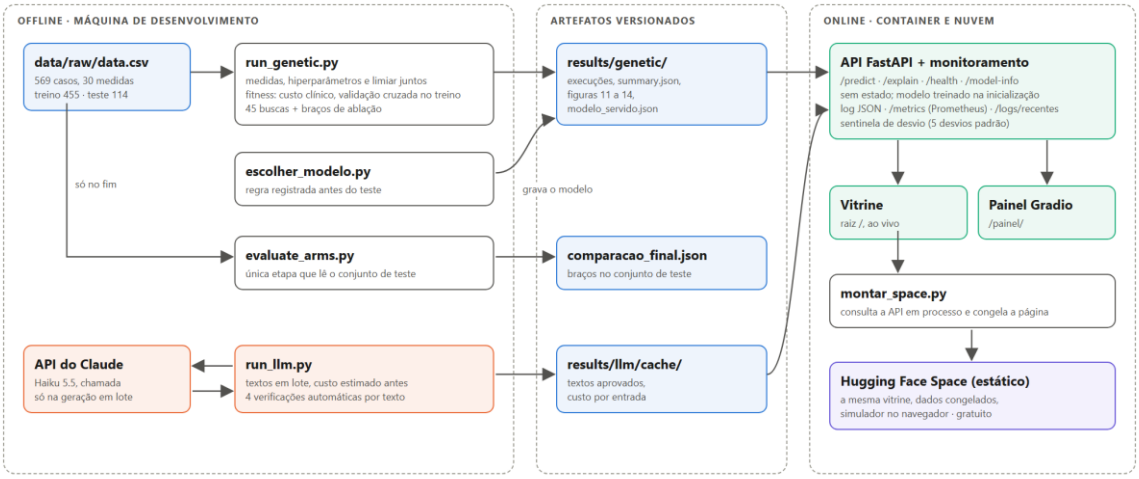
Por isso a Fase 2 otimiza medidas, hiperparâmetros e limiar conjuntamente, contra o custo clínico, por validação cruzada dentro do treino. Se um subconjunto menor de medidas preservar a segurança, a limitação mais séria admitida na Fase 1, a de que as 30 medidas exigem o trabalho de um patologista, deixa de ser retórica.

O código-fonte, os resultados e este relatório estão em https://github.com/gustavofks/ML-breast-cancer, com a Fase 1 congelada na tag `v1.0-fase1`. A vitrine interativa está publicada em https://huggingface.co/spaces/gustavofks/apoio-diagnostico-cancer-mama.

2 ARQUITETURA DA SOLUÇÃO

A arquitetura separa o que é caro ou pago do que é servido. As etapas pesadas (as buscas genéticas, que levam horas) e a única etapa que custa dinheiro (a geração de textos pela API do Claude) rodam uma vez, na máquina de desenvolvimento, e gravam arquivos versionados em `results/`. O serviço só lê esses arquivos: não precisa de modelo de linguagem, de chave de API nem de acesso à internet.

Figura 1 - Arquitetura da solução: etapas offline, artefatos versionados e serviço online



Fonte: elaborado pelo autor.

Essa fronteira é a garantia de reprodutibilidade, a mesma da Fase 1, em que o relatório HTML abre sem executar nada. Ela também é o que torna a publicação gratuita possível, como mostra a seção 8.

Tabela 1 - Módulos acrescentados na Fase 2

Pacote	**Responsabilidade**
`src/genetic/`	indivíduo tipado, reparo, operadores, fitness, laço evolutivo, experimentos e artefatos
`src/llm/`	provedores (Claude, Ollama e cache), prompts versionados, narrador e verificações de qualidade
`src/api/`	modelo servido, contribuições por caso, rotas, log estruturado e monitoramento
`src/ui/`	painel Gradio, vitrine interativa, simulador e casos congelados para a nuvem
`scripts/`	`run_genetic.py`, `evaluate_arms.py`, `escolher_modelo.py`, `build_genetic_report.py`, `run_llm.py` e `montar_space.py`

Fonte: elaborado pelo autor.

Os módulos da Fase 1 foram reaproveitados sem alteração. O algoritmo genético **importa** `build_pipeline` e a validação cruzada estratificada da Fase 1, em vez de recriá-

los. É isso que torna legítima a comparação entre modelo original e otimizado: os dois lados usam o mesmo código de treino e avaliação, e qualquer diferença vem da otimização, não de uma implementação divergente.

3 IMPLEMENTAÇÃO DO ALGORITMO GENÉTICO

Código correspondente: `src/genetic/`.

3.1 Codificação do indivíduo

O cromossomo é híbrido, em três segmentos, cada um com operador próprio. Escolhi uma representação tipada em vez de um vetor de bits homogêneo porque cada tipo de gene exige mutação e cruzamento diferentes, e forçar tudo em binário tornaria a decodificação obscura sem ganho de busca.

Tabela 2 - Segmentos do cromossomo

Segmento	**Conteúdo**	**Tipo**	**Domínio**
Máscara de medidas	30 genes, um por medida	binário	0 ou 1
Hiperparâmetros	específicos da família do modelo	misto	ver tabela seguinte
Limiar de decisão	um gene	real	de 0,05 a 0,95

Fonte: elaborado pelo autor.

Tabela 3 - Genes de hiperparâmetro por família de modelo

Família	**Genes**
Regressão Logística	`C` em escala logarítmica, de 0,001 a 100; `class_weight` entre nenhum e `balanced`
KNN	`n_neighbors` ímpar de 1 a 31; `weights` entre `uniform` e `distance`; `p` entre 1 e 2
Random Forest	`n_estimators` de 50 a 300; `max_depth` categórico; `min_samples_leaf` de 1 a 10; `max_features` categórico

Fonte: elaborado pelo autor.

Depois de todo operador, um reparo garante que nenhum indivíduo inválido chegue ao fitness: ao menos uma medida ativa, todo gene dentro do seu domínio. O reparo vale também para os segmentos congelados dos braços de ablação, e há teste automatizado que falha se um valor padrão da Fase 1 deixar de sobreviver a ele.

3.2 Função fitness

A função fitness traduz em número a política clínica da Fase 1, que estabeleceu o falso negativo como erro incomparavelmente mais grave:

```
custo   = (C_FN × falsos negativos + C_FP × falsos positivos) / n
```

```
fitness = 1 − custo / C_FN − α × (medidas ativas / 30)
```

Com `C_FN = 10`, `C_FP = 1` e `α = 0,05`. Dividir o custo por `C_FN` mantém o termo entre 0 e 1 qualquer que seja a razão escolhida. O termo de parcimônia é pequeno e serve de desempate: entre duas soluções igualmente seguras, prefere a que exige menos medidas do patologista. Os três pesos são parâmetros de configuração em `src/config.py`, nunca valores embutidos no código.

Duas regras metodológicas estão codificadas no módulo de fitness, e nenhuma é negociável. A primeira: o fitness é avaliado por validação cruzada estratificada de 5 partições **dentro do conjunto de treino**, com o limiar do próprio indivíduo aplicado sobre a probabilidade prevista fora de cada partição. É isso que corrige a fragilidade que a Fase 1 confessou ao escolher o limiar olhando o teste. A segunda: o conjunto de teste fica intocado até a comparação final. O pacote do algoritmo genético não importa nada que saiba abrir a base ou refazer a separação, e há teste que falha se qualquer caminho do fitness tocar dados de teste.

3.3 Operadores

Tabela 4 - Operadores genéticos e justificativas

Operador	**Escolha**	**Justificativa**
Seleção	torneio de tamanho *k*, com elitismo de 2 indivíduos	pressão seletiva ajustável por *k*; o elitismo impede que o melhor se perca
Cruzamento	uniforme na máscara, aritmético nos genes reais, troca nos categóricos	o cruzamento de ponto único criaria viés posicional entre medidas vizinhas, que aqui não têm relação de vizinhança
Mutação	troca de bit na máscara, perturbação gaussiana em escala logarítmica nos reais, sorteio nos categóricos	taxa por gene, configurável

Fonte: elaborado pelo autor.

3.4 Experimentos

O enunciado exige ao menos três configurações do algoritmo genético. São 4, porque a quarta responde uma pergunta que as outras não respondem: quanto o elitismo contribui de fato. As configurações variam apenas parâmetros do algoritmo, com a função fitness fixa, o que torna a comparação entre elas interpretável.

Tabela 5 - Configurações do algoritmo genético

Experimento	**População**	**Gerações**	**Mutação**	**Cruzamento**	**Torneio**	**Elitismo**
E1 base	40	30	0,10	0,80	3	2
E2	80	30	0,25	0,60	2	2

Experimento	População	Gerações	Mutação	Cruzamento	Torneio	Elitismo
exploração						
E3 explotação	20	60	0,05	0,90	5	2
E4 sem elitismo	40	30	0,10	0,80	3	0

Fonte: elaborado pelo autor.

Cada configuração rodou sobre as 3 famílias de modelo, com 3 sementes (42, 43, 44): algoritmo genético é estocástico, e resultado de execução única não é resultado. O total foi de 45 buscas e 59.069 avaliações de fitness.

3.5 Braços de ablação

Afirmar que "o algoritmo genético encontrou solução melhor" não prova nada, porque o ganho pode vir inteiramente do limiar. Os braços de comparação isolam as contribuições, congelando os segmentos que não otimizam nos valores padrão da Fase 1.

Tabela 6 - Braços de comparação

Braço	O que otimiza	Medidas	Limiar
Original	nada, o modelo padrão da Fase 1	30	0,50
Busca em grade	hiperparâmetros, como na Fase 1	30	0,50
AG, só limiar	o limiar	30	otimizado
AG, só features	a máscara de medidas	otimizadas	0,50
AG, só hiperparâmetros	os hiperparâmetros	30	0,50
AG, completo	os três segmentos	otimizadas	otimizado

Fonte: elaborado pelo autor.

Os braços de ablação rodam sobre a regressão logística, o modelo escolhido na Fase 1; as configurações do algoritmo rodam sobre as três famílias.

3.6 Execução

A Random Forest torna as buscas lentas: medi 2,54 s por avaliação de fitness contra 0,056 s da regressão logística. Duas medidas trouxeram as horas de execução para um patamar viável. O teto de `n_estimators` ficou em 300, exatamente o valor da Fase 1, para que o reparo não substituísse em silêncio o padrão do braço Original. E a população passou a ser avaliada em paralelo, com a garantia de que o resultado é idêntico ao da execução sequencial: o desempate entre indivíduos de mesmo fitness segue a ordem de primeira aparição, e não a ordem de término dos processos.

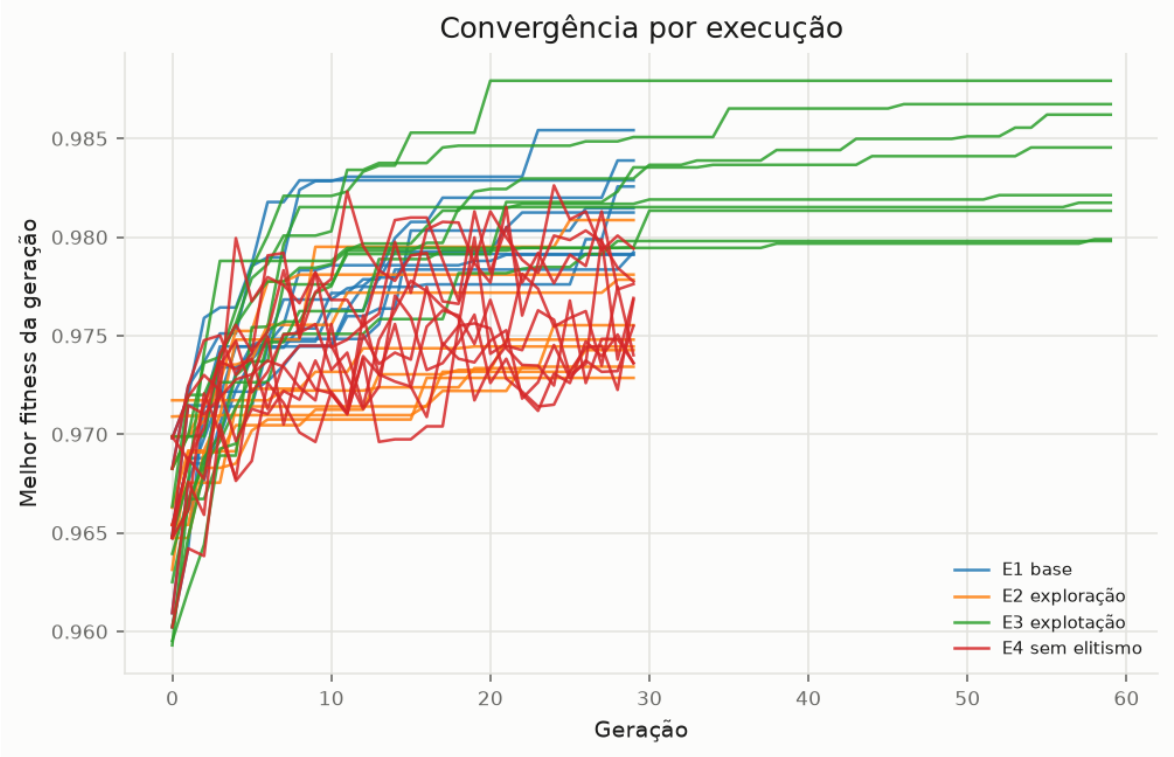
Uma reinicialização da máquina interrompeu a primeira execução completa. Daí nasceu a opção `--retomar`, que reaproveita as buscas já gravadas, identificadas por

experimento, família, semente, segmentos, política de custo e configuração inteira. O arquivo final de uma execução retomada é idêntico, byte a byte, ao de uma execução sem interrupção, porque cada busca nasce da própria semente.

4 RESULTADOS DA OTIMIZAÇÃO

4.1 Convergência

Figura 2 - Melhor fitness por geração em cada execução, colorido por configuração



Fonte: elaborado pelo autor.

A figura mostra o papel do elitismo com clareza. Nas configurações E1 a E3, o melhor fitness nunca cai, porque os dois melhores indivíduos passam intactos para a geração seguinte. Na E4, sem elitismo, as curvas oscilam: o melhor indivíduo é encontrado e perdido várias vezes, e a busca termina abaixo do pico que já tinha alcançado.

Tabela 7 - Resultado das configurações por família de modelo, braço completo, média de três sementes

Experimento	Família	Custo clínico médio	Desvio	Medidas (média)	Fitness médio
E1 base	KNN	0,1062	0,0108	5,00	0,9810
E1 base	Random Forest	0,1179	0,0075	5,00	0,9799
E1 base	Regressão Logística	0,0872	0,0063	4,33	0,9841
E2 exploração	KNN	0,1341	0,0315	7,00	0,9749
E2 exploração	Random Forest	0,1648	0,0177	6,00	0,9735
E2 exploração	Regressão Logística	0,0718	0,0127	8,33	0,9789

Experimento	Família	Custo clínico médio	Desvio	Medidas (média)	Fitness médio
E3 explotação	KNN	0,0901	0,0171	6,00	0,9810
E3 explotação	Random Forest	0,1062	0,0132	4,33	0,9822
E3 explotação	Regressão Logística	0,0637	0,0207	4,00	0,9870
E4 sem elitismo	KNN	0,1055	0,0200	6,33	0,9789
E4 sem elitismo	Random Forest	0,1355	0,0068	6,00	0,9764
E4 sem elitismo	Regressão Logística	0,0681	0,0082	6,67	0,9821

Fonte: elaborado pelo autor.

A regressão logística tem o menor custo em todas as configurações, confirmando a escolha da Fase 1 por um caminho independente. Entre as configurações, a de explotação (E3: população pequena, mais gerações, pressão seletiva alta) chegou ao menor custo médio na regressão logística, 0,0637. A diferença entre configurações, porém, é da ordem do desvio entre sementes, e não autoriza ordenar as configurações com segurança.

4.2 De onde vem o ganho

Tabela 8 - Braços de ablação na validação cruzada, regressão logística, média de três sementes

Braço	Custo clínico médio	Desvio	FN médio	FP médio	Medidas
só limiar	0,0791	0,0000	2,00	16,00	30,00
completo	0,0872	0,0063	2,00	19,67	4,33
só hiperparâmetros	0,1385	0,0000	6,00	3,00	30,00
só features	0,1626	0,0171	7,00	4,00	6,67

Fonte: elaborado pelo autor.

A tabela é o resultado mais importante do trabalho. Otimizar só o limiar produz o menor custo clínico, 0,0791, com as 30 medidas. Otimizar só a escolha de medidas, com o limiar fixo em 0,50, produz o maior, 0,1626. O braço completo, que otimiza tudo junto, fica entre os dois (0,0872), com custo próximo ao do limiar e com uma fração das medidas.

A leitura é que o ganho em segurança vem de **mover o ponto de operação**, e a seleção de medidas contribui com outra coisa: economia de medidas sem perder segurança. O algoritmo genético encontra modelos com quatro a seis medidas que mantêm o custo do modelo de 30, mas não modelos mais seguros por causa das medidas que escolhe.

4.3 Verificação de estabilidade

Com dezenas de milhares de avaliações, o algoritmo pode sobreajustar as partições específicas da validação cruzada. Para medir isso, os cinco melhores indivíduos de cada

execução foram reavaliados com outra semente de partição. Nas 225 reavaliações, a queda média de fitness foi de 0,0020 e a maior, 0,0088: sem sinal de sobreajuste às partições.

4.4 Seleção de medidas e convergência com a Fase 1

Figura 3 - Frequência com que cada medida entrou no melhor indivíduo



Fonte: elaborado pelo autor.

Tabela 9 - As dez medidas mais escolhidas pelo algoritmo genético

Medida	Frequência	Seleções	Posição no SHAP da Fase 1	No modelo servido
texture_worst	66,7%	26 de 39	1	sim
area_se	41,0%	16 de 39	12	sim
texture_mean	35,9%	14 de 39	14	sim
concavity_worst	33,3%	13 de 39	6	
radius_se	30,8%	12 de 39	2	
radius_worst	28,2%	11 de 39	5	
concave points_worst	28,2%	11 de 39	9	
fractal_dimension_se	25,6%	10 de 39	15	
concave points_mean	25,6%	10 de 39	4	sim
area_worst	23,1%	9 de 39	8	

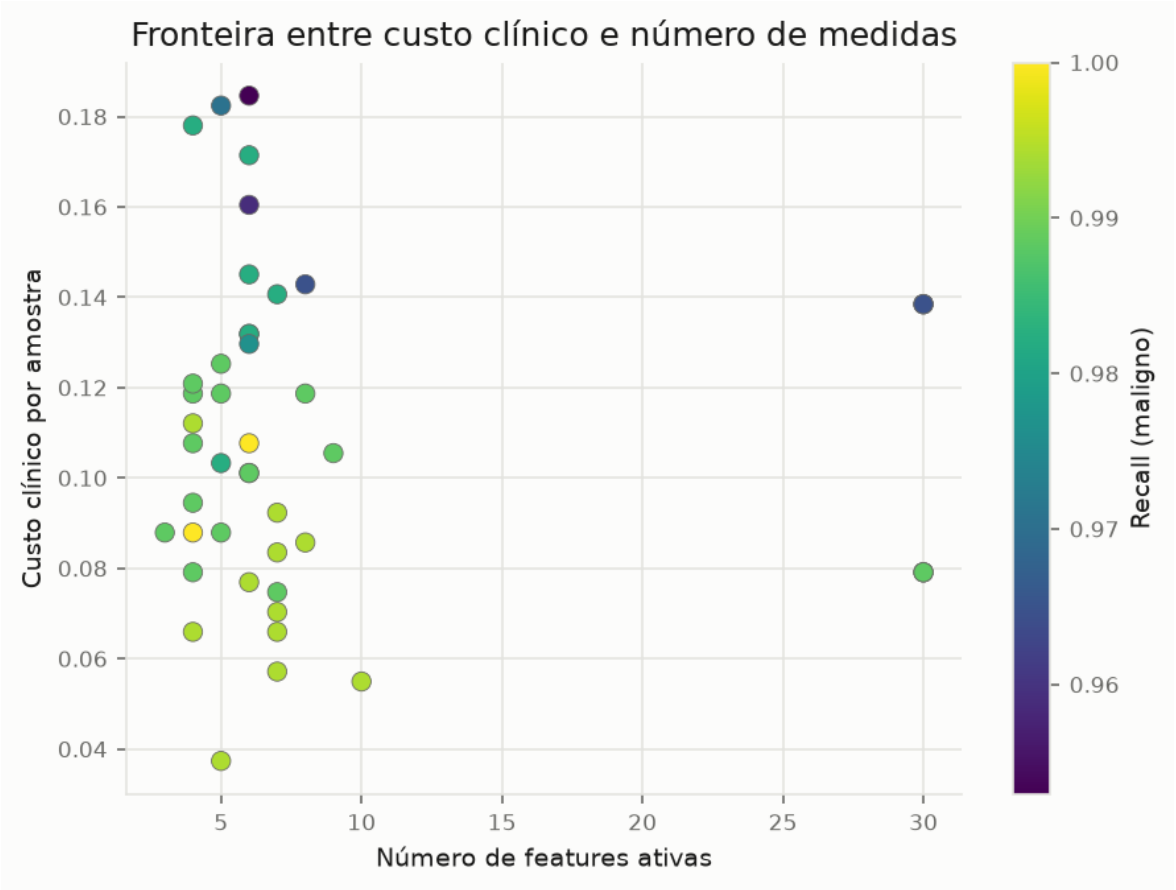
Fonte: elaborado pelo autor.

A Fase 1 mostrou que coeficientes, permutação e SHAP convergiam nas mesmas medidas. O algoritmo genético, por um quarto caminho, confirma parte disso:

`texture_worst`, a primeira do ranking SHAP, é também a mais escolhida. Mas a convergência não é completa, e a diferença é informativa: a seleção não é um recorte do topo do ranking SHAP. Medidas como `area_se` e `texture_mean`, nas posições 12 e 14 do SHAP, entram com frequência. Num modelo com poucas medidas, o que conta é a informação que cada uma acrescenta às outras escolhidas, e não o peso que ela teria ao lado das 30; o SHAP do modelo completo divide o crédito entre medidas redundantes, como raio, perímetro e área.

4.5 Fronteira entre custo e número de medidas

Figura 4 - Custo clínico contra número de medidas, um ponto por melhor indivíduo



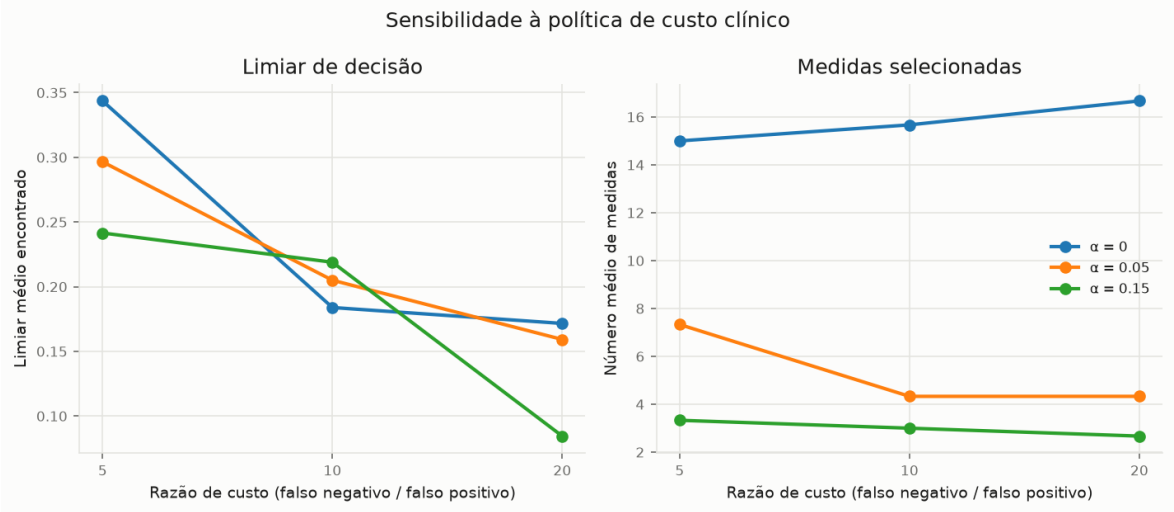
Fonte: elaborado pelo autor.

A fronteira confirma a leitura da seção 4.2. Os modelos de 30 medidas não são mais seguros que os de quatro a sete: os pontos de menor custo estão todos à esquerda. Reduzir de 30 para cerca de cinco medidas não custa segurança, e esse é o resultado prático da seleção.

4.6 Sensibilidade à política de custo

A razão de custo entre falso negativo e falso positivo é uma decisão de política clínica, não um parâmetro técnico, e a Fase 1 já defendeu que quem deve escolhê-la é a instituição de saúde. Uma varredura separada, de 27 buscas, mediu como a solução se desloca.

Figura 5 - Limiar encontrado e número de medidas conforme a razão de custo e a penalidade



Fonte: elaborado pelo autor.

Tabela 10 - Sensibilidade à política de custo clínico, configuração E1

Razão FN:FP	Penalidade	Limiar médio	Medidas	FN médio	FP médio
5:1	0,00	0,3436	15,00	2,00	7,33
10:1	0,00	0,1839	15,67	1,00	11,00
20:1	0,00	0,1716	16,67	1,00	9,67
5:1	0,05	0,2965	7,33	2,00	9,67
10:1	0,05	0,2050	4,33	2,00	19,67
20:1	0,05	0,1592	4,33	1,00	32,67
5:1	0,15	0,2415	3,33	2,00	24,00
10:1	0,15	0,2189	3,00	4,00	27,67
20:1	0,15	0,0845	2,67	2,00	58,67

Fonte: elaborado pelo autor.

O comportamento é o esperado e serve de prova de que o algoritmo responde à política: quanto mais pesa o falso negativo, mais baixo o limiar escolhido, e mais falsos positivos aceitos em troca. A penalidade por medida controla o tamanho do modelo quase sozinha. A curva entrega à instituição a informação para decidir, em vez de uma constante escolhida pelo cientista de dados.

5 COMPARAÇÃO ENTRE MODELOS ORIGINAIS E OTIMIZADOS

5.1 Regra de escolha registrada antes do teste

Escolher o modelo servido olhando o conjunto de teste repetiria a fragilidade da Fase 1. Por isso a regra foi gravada em `results/genetic/regra_do_modelo_servido.json`, em 11/10/2026, antes da comparação final: serve o indivíduo de maior fitness em validação cruzada entre todas as execuções, com uma trava de segurança. Se, no teste, o escolhido tiver mais falsos negativos que a referência da Fase 1, a API serve a referência. A trava não escolhe o melhor modelo pelo teste; só impede que um modelo pior em falsos negativos seja implantado.

O candidato foi AG, completo, E3 explotação, Regressão Logística, semente 44: fitness de 0,9879, custo clínico de 0,0374, 1 falso negativo e 7 falsos positivos na validação cruzada, queda de 0,0046 na verificação de estabilidade. É uma regressão logística com `C = 32,65`, limiar 0,2274 e 5 medidas: `texture_mean`, `concave points_mean`, `texture_se`, `area_se`, `texture_worst`.

5.2 Resultados no conjunto de teste

Este é o único uso dos 114 casos de teste em toda a Fase 2, feito por um script que nenhum outro passo importa.

Tabela 11 - Todos os braços nos casos de teste, ordenados por falsos negativos

Abordagem	**Medidas**	**Limiar**	**FN**	**FP**	**Recall**	**AUC**	**Custo clínico**
só hiperparâmetros	30	0,5000	1	1	0,9762	0,9977	0,0965
só limiar	30	0,1833	1	4	0,9762	0,9960	0,1228
Original	30	0,5000	3	1	0,9286	0,9960	0,2719
Busca em grade	30	0,5000	3	1	0,9286	0,9960	0,2719
completo (servido)	5	0,2274	3	4	0,9286	0,9901	0,2982
só features	6	0,5000	4	1	0,9048	0,9947	0,3596

Fonte: elaborado pelo autor.

A trava não disparou: o modelo servido tem 3 falsos negativos, como a referência. Os dois braços que só movem o ponto de operação chegam a 1 falso negativo. O braço "só hiperparâmetros" encontrou `class_weight = balanced`, que equivale a deslocar o limiar ao reponderar as classes, e teve o melhor resultado de todos: 1 falso negativo, 1 falso positivo e AUC de 0,9977. O braço "só limiar", com limiar de 0,1833, também teve 1 falso negativo, com 4 falsos positivos.

5.3 Leitura honesta do resultado

O conjunto de teste tem 42 casos malignos. Com o custo ponderado, cada falso negativo a mais vale 0,088 de custo clínico, e diferenças de um ou dois casos entre braços estão dentro do ruído de uma amostra desse tamanho. O teste não autoriza ordenar os braços; ele confirma o que a validação cruzada já mostrava, e é isso que importa.

Duas conclusões resistem ao ruído. A primeira: o ganho em segurança vem do ponto de operação, e os dois braços que só o ajustam são os únicos que reduzem os falsos negativos. A segunda: o modelo servido usa 5 medidas em vez de 30 sem perder segurança em relação à referência, o que responde em parte à limitação que a Fase 1 admitiu sobre o trabalho do patologista.

O modelo servido não é o melhor do teste, e mantê-lo é deliberado. Trocar pelo braço de melhor resultado no teste seria escolher pelo teste, exatamente o que a regra existe para impedir. A troca legítima é outra: registrar uma nova regra que favoreça o ponto de operação e validá-la em dados novos.

6 INTEGRAÇÃO COM MODELO DE LINGUAGEM

Código correspondente: `src/llm/` e `scripts/run_llm.py`.

6.1 Abordagem

Os textos são gerados uma vez, em lote, pelo Claude Haiku 5.5 (`claude-haiku-5-5`) via API da Anthropic, e gravados num cache versionado em `results/llm/cache/`. A API e as telas leem o cache; nenhuma delas chama o modelo de linguagem. A decisão inicial era um modelo local via Ollama, e foi revista: o Claude escreve melhor em português, dispensa instalação, e o lote inteiro custa centavos. O Ollama continua como alternativa gratuita, a uma opção de linha de comando.

Toda a camada fala com um `Protocol` de provedor, com um único método de geração. Trocar o modelo é mudar uma linha de configuração, e o nome do modelo entra na chave do cache: texto gerado por um modelo nunca é servido como se fosse de outro. É também essa interface que prepara a base para o Módulo 3: os dados textuais que vierem entram como mais um tipo de evidência e mais um template, sem tocar no provedor, no cache ou nas verificações.

O custo é travado no código. Antes da primeira chamada, o lote inteiro é estimado no pior caso e barrado se passar do orçamento de US$ 1,00; cada chamada confere o saldo de novo; e texto já gerado nunca é pago outra vez. O lote real de 118 textos consumiu 78.662 tokens de entrada e 76.344 de saída, a US$ 0,10 e US$ 0,50 por milhão, e custou **US$ 0,046**.

Tabela 12 - Produtos de texto

Texto	Função	Evidência entregue ao modelo
Explicação por paciente	narra a previsão de um caso para o médico	probabilidade, limiar, decisão e as contribuições do caso
Resumo para a gestão clínica	transforma métricas em orientações acionáveis	as métricas do modelo em `results/metrics.json`
Nota de mudança	explica o que o algoritmo genético alterou	o `summary.json` do algoritmo genético

Fonte: elaborado pelo autor.

6.2 Técnicas de prompt engineering

Os prompts vivem em `src/llm/prompts.py`, com nome e versão, para que o relatório possa citar qual prompt produziu qual texto. As técnicas aplicadas, cada uma visível no texto dos templates:

- **papel e fronteira no prompt de sistema:** apoio à decisão, nunca diagnóstico, com a obrigação de dizer que a leitura não substitui o médico;
- **evidência em JSON, não em prosa:** número em prosa convida o modelo a reescrever, número em JSON convida a copiar;
- **exemplo resolvido**, para fixar formato e tom;
- **contrato de saída explícito**, com seções nomeadas e limite de palavras;
- **proibição de números novos:** usar apenas os valores fornecidos, sem calcular nem arredondar.

O trecho a seguir é o núcleo do prompt da explicação por paciente, na versão servida:

```
Regras:
- Use apenas os valores fornecidos acima. Não calcule, não arredonde
  e não estime nenhum número ausente da evidência.
- Cite nominalmente as três medidas de maior contribuição.
- Escreva no máximo 220 palavras.
- Use exatamente estas seções, nesta ordem, como títulos de nível 2:
  Leitura do caso, Medidas que mais pesaram, Ressalva.
```

6.3 Avaliação automática da qualidade

O enunciado pede que a qualidade das interpretações seja avaliada. Aqui ela é medida, não afirmada: cada texto passa por quatro verificações automáticas, e texto reprovado nunca chega à tela.

Tabela 13 - Verificações automáticas e resultado do lote

Verificação	Mecanismo	O que detecta	Aprovados
fidelidade	todo número do texto existe na evidência, com sinal	alucinação de estatística	116 de 116
seguranca	padrões de afirmação diagnóstica, com negação; ressalva obrigatória	texto que se passaria por laudo	116 de 116
cobertura	as três medidas de maior contribuição são citadas	explicação genérica	116 de 116
formato	seções do contrato presentes e limite de palavras	saída fora do contrato	116 de 116

Fonte: elaborado pelo autor.

Os 116 textos do lote passaram nas quatro verificações. As próprias verificações, porém, precisaram ser corrigidas antes disso, e os defeitos foram encontrados nos textos reais. A verificação de fidelidade ignorava o sinal: uma contribuição de −5,798 citada como 5,798 passaria. A de segurança acusou nove falsos positivos no primeiro lote: ela removia acentos antes de buscar os padrões proibidos, ignorava negações como "não confirma" e não reconhecia "nem substitui" como ressalva. Cada frase real que causou um erro virou teste automatizado.

6.4 Comparação entre versões de prompt

Tabela 14 - Versões de prompt e aprovação na verificação de formato

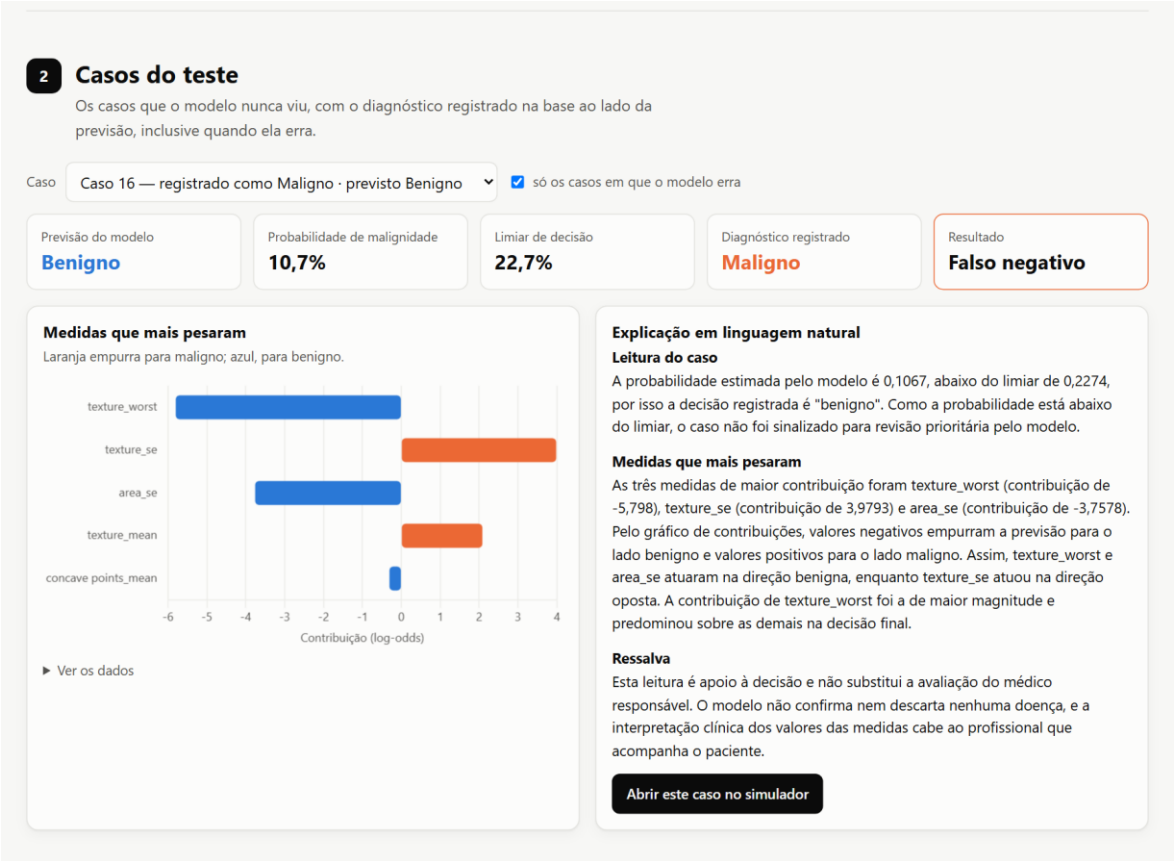
Prompt	Textos gerados	Palavras (média)	Aprovados no formato
`explicacao_paciente_v1`	1	173	1 de 1
`explicacao_paciente_v2`	114	131	114 de 114
`nota_de_mudanca_v1`	1	229	0 de 1
`nota_de_mudanca_v2`	1	194	1 de 1

Fonte: elaborado pelo autor.

A versão 1 da explicação por paciente não tinha exemplo resolvido nem limite explícito de palavras; gerou 173 palavras no caso de comparação e passou nas verificações. A versão 2, com exemplo e contrato fechado, produziu textos de 87 a 171 palavras, com média de 131, todos dentro do limite de 220. A diferença mais clara apareceu na nota de mudança: a versão 1 pedia exatamente o limite que o verificador aplica, e o modelo passou dele, com 229 palavras, e foi reprovada no formato. A versão 2 pede no máximo 180 palavras, uma folga em relação ao limite, e gerou 194. A lição é que o modelo trata o limite pedido como alvo aproximado, e o prompt precisa pedir menos do que o contrato tolera.

6.5 Exemplo e avaliação humana

Figura 6 - Vitrine ao vivo: um falso negativo do modelo servido, com as contribuições e a explicação gerada



Fonte: elaborado pelo autor.

A figura mostra o caso mais importante de todos, um tumor maligno classificado como benigno. O texto cita os números certos, atribui a cada medida o sentido correto e não esconde a decisão, mas também não tem como saber que ela está errada. É o mesmo limite que a análise SHAP do falso negativo mostrou na Fase 1, agora em linguagem de médico: a explicação descreve o que o modelo viu, não o que o modelo deixou de ver.

As verificações automáticas conferem fatos, não utilidade. Por isso uma rubrica humana complementa a avaliação: 7 textos escolhidos para cobrir os erros do modelo, casos perto do limiar e os dois textos de gestão, pontuados de 1 a 5 em correção, utilidade, clareza e segurança.

Tabela 15 - Rubrica humana, de 1 a 5, avaliação subjetiva de um único avaliador, o autor

Texto	Correção	Utilidade	Clareza	Segurança	Média
Caso 16, falso negativo	3	2	5	4	3,5
Caso 24, falso negativo	3	2	5	4	3,5
Caso 4, falso positivo	3	2	5	4	3,5

Texto	Correção	Utilidade	Clareza	Segurança	Média
Caso 92, benigno perto do limiar	4	3	5	4	4,0
Caso 3, maligno perto do limiar	4	4	5	5	4,5
Resumo para a gestão	4	4	5	5	4,5
Nota de mudança	5	4	4	5	4,5

Fonte: elaborado pelo autor.

A média geral foi 4,0, e a menor nota, 2. Por critério, a clareza teve média 4,9, a segurança 4,4, a correção 3,7 e a utilidade 3,0, a mais baixa. A queda de utilidade se concentra nos erros do modelo: nos 3 casos de erro, ela foi de 2,0. A observação registrada nesses casos é a mesma: o texto descreve a previsão com correção, mas não sinaliza que ela diverge do diagnóstico registrado, e uma classificação benigna errada pode transmitir falsa segurança.

Parte dessa limitação é estrutural, e não do prompt. Em uso real, o diagnóstico de referência não existe no momento da previsão, e a camada de linguagem recebe apenas o que o modelo viu. Os dois falsos negativos avaliados têm probabilidade bem abaixo do limiar, e nada na evidência permitiria ao texto suspeitar deles; quem os pega é o médico, e é por isso que o sistema é de apoio. A outra parte é corrigível, e a avaliação apontou onde: nos casos perto do limiar, o texto deveria tratar a proximidade como motivo explícito de cautela e recomendar revisão. É o próximo passo concreto da camada de linguagem: uma versão 3 do prompt que receba a distância ao limiar como evidência e peça essa recomendação quando ela for pequena, medida nas mesmas verificações automáticas e na mesma rubrica.

A ressalva é explícita: é a leitura de um único avaliador, sem formação médica, e serve para registrar a percepção de utilidade, não para validá-la clinicamente.

7 SERVIÇO, MONITORAMENTO E ESCALABILIDADE

Código correspondente: `src/api/` e `src/ui/`.

7.1 API

Tabela 16 - Rotas da API

Rota	Função
`GET /health`	vivacidade e confirmação de que o modelo carregou
`GET /model-info`	modelo servido, medidas, limiar, origem e commit
`POST /predict`	probabilidade, classe e limiar a partir das medidas
`POST /explain`	a previsão, as contribuições do caso e o texto em linguagem natural

Rota	Função
`GET /metrics`	contadores, latências e alertas de desvio no formato Prometheus
`GET /logs/recentes`	os eventos de log mais recentes, para a vitrine mostrar na tela

Fonte: elaborado pelo autor.

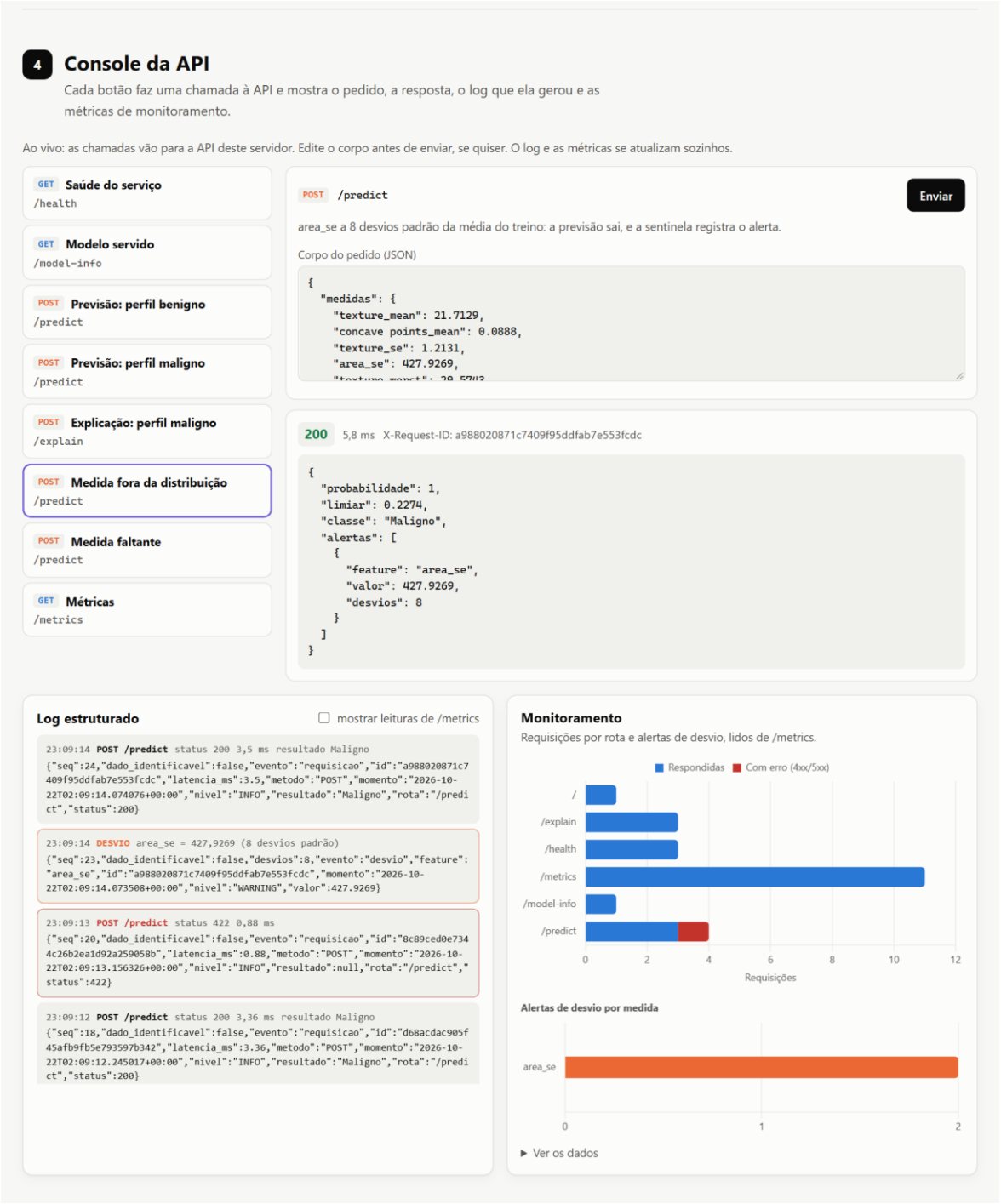
A documentação interativa da API é gerada pela própria FastAPI em `/docs`. O modelo servido é **treinado na inicialização**, a partir da especificação gravada, e não carregado de um arquivo binário: o formato do scikit-learn só abre na mesma versão que o gravou, e treinar 455 amostras custa menos de um segundo. A entrada é validada: medida faltante, desconhecida ou não finita é recusada com status 422 e uma mensagem que nomeia a medida, em vez de ignorada em silêncio.

7.2 Logging e monitoramento

Cada requisição gera uma linha de log em JSON, com identificador, rota, método, status, latência e resultado. As entradas são medidas morfológicas anônimas, e cada linha declara isso num campo próprio: a ausência de dado identificável é afirmada, não presumida. A rota `/metrics` expõe contadores por rota e status, latências e alertas no formato que o Prometheus coleta sem configuração.

A sentinela de desvio implementa a mitigação que a Fase 1 prescreveu para a degradação silenciosa. Ela compara cada medida recebida com a distribuição do treino e registra alerta quando alguma foge por mais de 5 desvios padrão. O valor foi medido no próprio treino: com três desvios, 13% dos casos legítimos dispariam alerta, e alerta que dispara sempre vira ruído ignorado. A sentinela registra, não bloqueia: quem decide se confia no resultado é o médico.

Figura 7 - Console da vitrine ao vivo: medida fora da distribuição, alerta no log e métricas atualizadas



Fonte: elaborado pelo autor.

7.3 Escalabilidade

O enunciado pede recursos de escalabilidade automática. O serviço está **preparado** para escalar: não guarda estado além de contadores em memória, carrega o modelo uma vez na inicialização, expõe verificação de saúde e métricas, e roda num container único. Qualquer réplica responde igual a qualquer outra, que é a condição para escalar horizontalmente.

A escala automática em si exige uma plataforma que a execute, e a camada gratuita usada aqui não a oferece; o relatório não vai fingir que oferece. A arquitetura que escalaria de verdade é direta: o mesmo container em várias réplicas atrás de um balanceador, num orquestrador como Kubernetes ou Cloud Run, com regra de escala pela latência ou pelo uso de CPU que `/metrics` já expõe, e a verificação de saúde usada pelo balanceador para retirar réplicas com problema. O ajuste necessário no código seria um só: os contadores de `/metrics` passariam a ser somados pelo coletor do Prometheus entre as réplicas.

7.4 Vitrine e painel

A raiz do servidor abre a vitrine, uma página de apresentação com cinco capítulos: como o modelo foi escolhido, os casos do teste, um simulador de medidas, um console da API e a qualidade dos textos. Servida pelo container, ela detecta a API e chama as rotas de verdade: o console mostra o pedido, a resposta, o log que ela gerou e as métricas, na tela, em vez de no terminal. O painel Gradio, que consome a API por HTTP, continua em `/painel/`.

8 IMPLANTAÇÃO EM NUVEM

A vitrine está publicada no Hugging Face Spaces, em https://huggingface.co/spaces/gustavofks/apoio-diagnostico-cancer-mama. O plano era publicar o container inteiro, com API e painel. Durante a implantação, o Hugging Face passou a cobrar de contas pessoais pelos Spaces que executam código, nos formatos Docker e Gradio; só o Space estático continua gratuito. Como o projeto não aceita custo recorrente, a publicação foi adaptada.

O Space estático não executa nada no servidor, então a vitrine roda lá em modo congelado. Na montagem, `scripts/montar_space.py` sobe a API em processo e congela o que ela responde: o `/explain` de cada um dos 114 casos de teste e uma sessão gravada do console, com respostas, log e métricas reais. Os números da página são os da API, não uma conta paralela. O simulador de medidas, que no container chama `/explain`, no Space refaz no navegador a conta da regressão logística servida. Isso só é legítimo porque é verificado: os testes comparam a conta do navegador com a da API nos 114 casos, com diferença abaixo de 10^{-9}.

A infraestrutura é descrita em código: o `Dockerfile` tem estágios para cada uso (pipeline tabular, visão, API e geração de textos), e o pacote do Space é gerado por script, com o cartão de configuração, a licença e o commit de origem. A limitação é que o

provisionamento do Space em si foi feito pela interface do Hugging Face, e não por uma ferramenta de infraestrutura como código.

9 DESAFIOS ENFRENTADOS E SOLUÇÕES

Tabela 17 - Principais desafios e como foram resolvidos

Desafio	**Solução**
Random Forest 45 vezes mais lenta que a regressão logística por avaliação	teto de 300 árvores, o valor da Fase 1, e avaliação da população em paralelo, com resultado idêntico ao sequencial
Reinicialização da máquina no meio da execução completa	opção `--retomar`, com arquivo final idêntico byte a byte ao de uma execução sem interrupção, e recusa de sobrescrever resultados existentes
A varredura de custo sobrescrevia o arquivo dos experimentos	destino próprio para a varredura e teste que impede a colisão
Política do Windows bloqueou as bibliotecas compiladas do ambiente virtual	a suíte de testes passou a rodar no container do próprio projeto
A API do Claude recusa o parâmetro de temperatura no modelo usado	geração com esforço `low` e determinismo garantido pelo cache, não pela amostragem
Verificador de segurança com nove falsos positivos nos textos reais	busca com acentos, tratamento de negação e ressalvas equivalentes, com as frases reais como testes
Verificador de fidelidade ignorava o sinal dos números	números extraídos com sinal e conferidos pelo sinal quando ele existe
Nota de mudança acima do limite de palavras	segunda versão do prompt, pedindo menos do que o contrato tolera
Hugging Face passou a cobrar pelos Spaces que executam código	vitrine em modo estático, com simulador verificado contra a API
O Git Bash reescreve o caminho do volume do Docker	documentação do comando correto e indicador do modelo servido na vitrine

Fonte: elaborado pelo autor.

10 REPRODUTIBILIDADE E TESTES

Semente fixa em toda etapa estocástica, dependências com versão travada e resultados versionados. O clone traz todos os artefatos da Fase 2, então os resultados podem ser conferidos sem executar nada. Para executar:

```
docker build --target api -t ml-breast-cancer:api .
docker run --rm -p 8000:8000 -v "$(pwd)/results:/app/results" ml-breast-
cancer:api
```

A vitrine fica em http://localhost:8000. Para refazer as etapas offline:

```
python scripts/run_genetic.py --retomar      # buscas do algoritmo genético
python scripts/evaluate_arms.py              # comparação final, a única que
usa o teste
python scripts/build_genetic_report.py       # summary.json e figuras 11 a 14
python scripts/run_llm.py --casos todos --estimar   # custo do lote, sem
gastar
```

A suíte tem mais de 400 testes automatizados em Python, um arquivo por módulo, mais os testes da lógica da vitrine em JavaScript. Os 86 testes da Fase 1 continuam passando

sem alteração. Os testes preferem afirmar propriedades a valores: o fitness nunca toca o teste, a execução retomada é idêntica à contínua, o simulador reproduz a API, o log da montagem não vaza para o monitoramento servido, e texto reprovado nunca chega à tela.

11 CONSIDERAÇÕES FINAIS

A Fase 2 aplicou o algoritmo genético onde ele é insubstituível, no espaço combinatório que a Fase 1 deixou aberto, e desenhou os experimentos para que o resultado pudesse contrariar a expectativa. Ele contrariou. A expectativa era que escolher medidas tornasse o modelo mais seguro; os braços de ablação mostraram que a segurança vem do ponto de operação, e que a escolha de medidas entrega outra coisa, igualmente útil: o mesmo nível de segurança com uma fração do trabalho do patologista.

A camada de linguagem transforma cada previsão num texto que o médico lê, e a qualidade desse texto é verificada por quatro critérios automáticos e uma rubrica humana. O serviço publicado é reprodutível e offline por construção, e a nuvem gratuita foi aproveitada no limite do que ela permite, com a limitação declarada.

O caso do falso negativo explicado pelo modelo de linguagem resume a posição do projeto: o sistema descreve com precisão o que viu, e não tem como saber o que deixou de ver. Por isso ele é ferramenta de triagem e segunda opinião, e o diagnóstico continua sendo do médico.

REFERÊNCIAS

ANTHROPIC. **Claude API documentation**. Disponível em: https://docs.anthropic.com. Acesso em: 2026.

GOLDBERG, D. E. **Genetic algorithms in search, optimization, and machine learning**. Reading: Addison-Wesley, 1989.

HOLLAND, J. H. **Adaptation in natural and artificial systems**. Ann Arbor: University of Michigan Press, 1975.

HUGGING FACE. **Spaces documentation**. Disponível em: https://huggingface.co/docs/hub/spaces. Acesso em: 2026.

LUNDBERG, S. M.; LEE, S.-I. A unified approach to interpreting model predictions. In: **Advances in Neural Information Processing Systems (NIPS)**, 2017.

PEDREGOSA, F. et al. Scikit-learn: machine learning in Python. **Journal of Machine Learning Research**, v. 12, 2011.

UCI MACHINE LEARNING REPOSITORY. **Breast Cancer Wisconsin (Diagnostic) Data Set**. Disponível em: https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data. Acesso em: 2026.